

apresentacao.py x

```
1 # Iniciação em Programação em Python
2
3 # Apresentação do Curso e dos Membros
4
5 membros = [
6     "Guilherme Lopes",
7     "Guilherme Shimano",
8     "José Hernandez",
9     "Vitor Teodoro",
10    "Lucas Nakamura"
11 ]
12 dias_de_aula = ["31/07", "07/08", "14/08", "21/08"]
13 duracao_horas = 3 # 13:45 - 16:30
14 material_base_usado = "Material do Professor Malbarbo em malbarbo.pro.br!"
15
```

dia_1.py x

Dia 1 -- Introdução à Python

'''

Como instalar Python?

- Python é um software livre e pode ser baixado e instalado em:

<https://www.python.org/>

- Além do interpretador da linguagem, Python conta com um ambiente de desenvolvimento chamado IDLE!

- Em um sistema Linux, muito provavelmente o Python já vêm instalado por padrão, basta apenas instalar o IDLE. Em um sistema baseado em Debian, basta usar o comando:

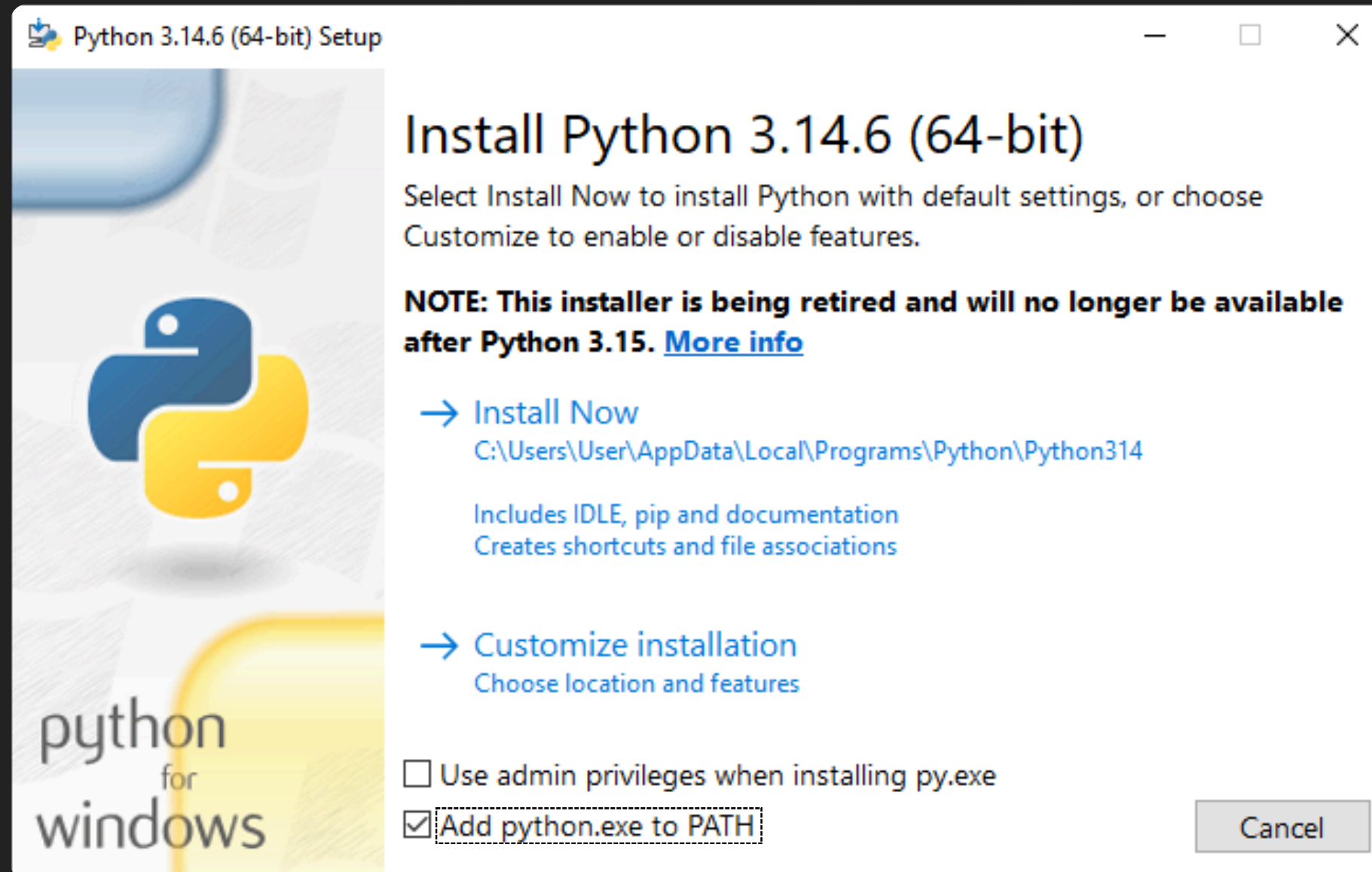
```
$ sudo apt install idle
```

'''

instalando_python.py x

'''

- Para se instalar no Windows, é importante marcar a opção "Add python.exe to PATH"

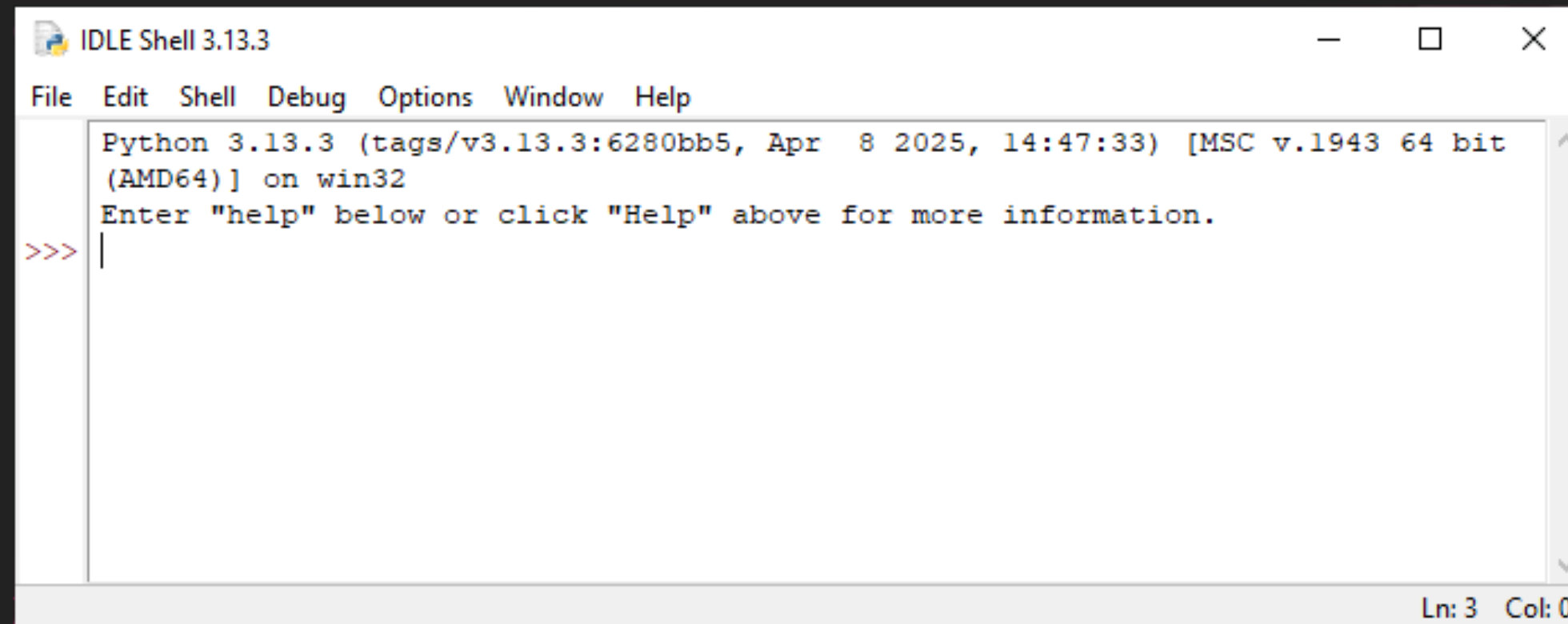


'''

instalando_python.py x

'''

Ao iniciar o IDLE, é aberta a seguinte janela:



Essa é a chamada janela de interações (ou REPL), usada para testar pequenos trechos de códigos.

O símbolo >>> é chamado de *prompt* e indica que o interpretador está pronto.

'''



instalando_python.py x

'''

Como funciona Python?

- Python é uma linguagem interpretada:

- Durante a execução, um interpretador Python “traduz” em tempo real o código para linguagem de máquina para ser executado.

- Linguagens interpretadas costumam ser mais lentas que linguagens compiladas (traduzidas antes da execução para um arquivo executável).

- Entretanto, é mais rápido de se trabalhar com elas, porque podemos escrever e executar imediatamente.

- As interações no Python seguem a seguinte lógica:

1. Escrevemos o código

2. O código é avaliado

3. O resultado é exibido na tela

4. O processo se repete

'''



instalando_python.py x

1
2 '''

3 Por que usar Python?

4 - Sintaxe (forma que escrevemos o código) simples.

5 - Podemos focar mais na lógica do programa do que no “como fazer?”.

6 - Grande quantidade de recursos prontos

7 - Não é necessário se preocupar com criar diversas funcionalidades úteis
8 por conta própria, elas já estão prontas.

9 - É muito popular

10 - Grande quantidade de materiais extras para aprender e muitas

11 bibliotecas (conjunto de funcionalidades extras) já implementadas pela
12 comunidade.

13 '''
14
15

Tipos de Dados

'''

O que é um "tipo de dado"?

- São as formas que podemos representar uma informação em nosso programa.
- Cada tipo tem uma forma de representação, valores válidos disponíveis e operações relacionadas a eles.
- Inicialmente trabalharemos com os chamados "tipos primitivos".
 - Os tipos primitivos são os tipos de dados mais básicos que a linguagem nos fornece.

'''



tipos_de_dados.py x

1
2
3 # No Python temos os seguintes tipos de dados:

4 tipos_de_dados = {

5 "int": "Números inteiros",

6 "float": "Números com ponto flutuante (aproximações de números reais)",

7 "string": "Cadeias de caracteres",

8 "bool": "Valores booleanos (verdadeiro ou falso)"

9 }

10
11 # Na aula de hoje trataremos dos três primeiros, e deixaremos o papo sobre

12 # booleanos para a próxima aula.
13
14
15

tipos_de_dados.py x

Dados numéricos (int e float)

'''

Como dito anteriormente, usamos o tipo primitivo int para representar números inteiros, isto é, números positivos e negativos, sem vírgula.

- Ex: 1, -4, 67, -9926

Além do int, usamos o float quando queremos representar números reais (números que possuem vírgula).

- Exemplos: 6.9, 0.125e3 (Notação científica, $0.125 * 10^3$)

'''

tipos_de_dados.py x

```
1
2 '''
3 Como são números, é fácil deduzir que as principais operações desses tipos
4 são as mesmas operações que temos na matemática
5 '''
6 # Operações em int e float
7 >>> # Soma e subtração
8 >>> 5 + 10
9 15
10 >>> 6 - 7
11 -1
12
13
14
15 >>> # Multiplicação e divisão
16 >>> 4 * 3
17 12
18 >>> 7 / 2
19 3.5
20 >>> 8 / 2
21 4.0
```



tipos_de_dados.py x

```
1
2
3 '''
4 - Como uma divisão pode resultar um número não inteiro, o resultado dela é
5   dado como float.
6 - O float não consegue representar com precisão todos os números reais,
7   devido a uma limitação dos próprios computadores.
8 - Sabendo disso, em alguns casos, os resultados em ponto flutuante podem
9   parecer meio estranhos e até mesmo imprecisos.
10 - Toda operação que usa ao menos um float, resultam sempre em um float.
11 '''
12
13
14
15
```

tipos_de_dados.py x

1

2 # Outras operações úteis com números são:

3 >>> # Piso da divisão

4 >>> 464 // 7

5 67

6 >>> 3 // 2.0

7 1.0

8 >>> # Módulo

9 >>> 14 % 3

10 2

11 >>> -14 % 3

12 -1

13 >>> 5 % 1.3

14 1.0999999999999999

15

>>> # Exponenciação

>>> 4 ** 2

16

>>> 8.0 ** (1/3)

2.0

tipos_de_dados.py x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

```
'''
```

Quando realizamos múltiplas operações em uma única linha, elas são realizadas seguindo a ordem de precedência que já estamos acostumados, o PEMDAS:

- Operações dentro de Parênteses

- Exponenciação

- Multiplicação e Divisão

- Adição e Subtração

```
'''
```

```
>>> 7 + 8 ** 2 - 2
```

```
69
```

```
>>> (7 + 8) ** (2 - 2)
```

```
1
```


tipos_de_dados.py x

Strings

'''

- Strings são a representação de cadeias de caracteres (um conjunto de letra, números e símbolos)
- Seu principal uso é para representar textos, mas as strings não se limitam a apenas isso
- Toda string em Python é escrita entre aspas simples/aspóstrofo (') ou aspas duplas (")

'''

string1 = "Isso é uma string"

string2 = 'Isso é outra string!'

string3 = '45%\$2321' # Isso também é uma string!

tipos_de_dados.py x

```
1  '''
2  Como os tipos de dados numéricos, também é possível realizar algumas
3  operações com strings.
4  '''
5  # Operações com strings:
6  >>> # Concatenação
7  >>> 'a' + 'b'
8  'ab'
9  >>> # Repetição
10 >>> 'papai' * 3
11 'papoipapoipapai'
12 >>> 'papai' * 0
13 ''
14 >>> 'papai' * -4
15 ''
```

```
>>> # Quantidade de caracteres
>>> len("Paralelepipedo")
14
>>> # Conversão para maiúscula
>>> "Jose".upper() # Ou str.upper("Jose")
"JOSE"
>>> # Conversão para minúscula
>>> "JoAo".lower() # Ou str.lower("JoAo")
"joao"
```

tipos_de_dados.py x

```
1
2 # Substrings
3 >>> # Indexação de caractere
4 >>> # 0 primeiro caractere tem índice 0
5 >>> 'Computação'[0]
6 'C'
7 >>> 'Computação'[5]
8 't'
9 >>> # Acesso de índice fora do intervalo
10 >>> 'Três'[4]
11 Traceback (most recent call last):
12   File "<stdin>", line 1, in <module>
13 IndexError: string index out of range
14
15
```




tipos_de_dados.py x

1 >>> # Substring do início até 4 - 1

2 >>> 'Alan Turing'[:4]

3 'Alan'

4 >>> # Substring de 5 até o final

5 >>> 'Alan Turing'[5:]

6 'Turing'

7 >>> # Substring de 3 até 7 - 1

8 >>> 'Alan Turing'[3:7]

9 'n Tu'

10

11 '''

12 Nota-se que a contagem das posições começa no 0, ao invés do 1, por isso

13 precisamos pegar a posição que queremos menos 1, ao usar os índices.

14 '''

15

Conversão entre tipos

'''

- Podemos converter dados que estão representados em um tipo para outro
- Para fazermos isso, basta digitar o nome do novo tipo e colocar o valor que queremos entre parênteses
- Tome cuidado usando a conversão de tipos, pois caso faça uma conversão inválida, um erro será disparado

'''

>>> # Conversão de int para str

>>> str(412)

'412'

>>> # Conversão float para str

>>> str(89.2)

'89.2'

>>> # Concatenação de str e int

>>> 'Idade: ' + str(20)

'Idade: 20'

>>> # Conversão de str para int

>>> int('10')

10

Variáveis

'''

- Variável é o nome que damos para uma região da memória que utilizamos para armazenar um dado
- Toda variável possui um tipo que indica o tipo de dado que pode ser armazenado dentro dela
- Para facilitar o entendimento, imagine que uma variável seja uma caixa com um rótulo que descreva o que tem dentro dela (nome da variável). Além disso, essa caixa tem um formato específico (tipo da variável) que garante que só objetos (dados) do mesmo tipo possam ser guardados dentro dela
- Sendo assim, uma variável do tipo int só pode guardar dados do tipo int, variáveis do tipo string só podem guardar dados do tipo string, etc.

'''



variaveis.py x

```
1
2 '''
3     - O nome da variável, em geral, pode ser qualquer coisa que possa se digitar
4       em um teclado. Entretanto é comum que linguagens definam regras que
5       restringem um pouco isso.
6     - Quando você quiser um dado armazenado em uma variável, basta simplesmente
7       usar o nome dela no lugar do dado.
8     - Para criar uma variável precisamos declarar ela e depois inicializamos com
9       um valor.
10      - Declarar: informar a linguagem que você quer que uma nova variável de
11        um determinado tipo exista.
12      - Inicializar: inserir o primeiro tipo de uma variável, tornando ela
13        usável
14 '''
15
```



variaveis.py x

1

2

3 >>> # Declarar (nome: tipo)

4 >>> a: int

5 >>> # Inicializar (nome = dado)

6 >>> a = 10

7 >>> # Chamar a variável

8 >>> a

9 10

10 >>> # A maioria das linguagens permitem declarar e inicializar ao mesmo tempo

11 >>> nome: str = "Guilherme"

12 >>> nome

13 "Guilherme"

14

15

variaveis.py x

```
1
2 '''
3 - Observe que o sinal "=", nesse caso, não significa "igual", ele indica a
4   operação de atribuição, ou seja, a operação de inserir um novo dado em uma
5   variável
6 - Em Python, a declaração do tipo da variável é opcional, podemos apenas
7   fazer a inicialização direto com nome = dado, e a própria linguagem já
8   define o tipo da variável sendo o mesmo do tipo do dado informado
9 - Entretanto, como estamos começando com o básico da programação, e isso é
10  algo obrigatório em várias outras linguagens, adotaremos essa prática nos
11  exemplos e recomendamos que vocês também usem nos exercícios para evitar
12  confusões
13 '''
14
15
```

variaveis.py x

```
1
2 '''
3 Como dito anteriormente, a maioria das linguagens colocam regras sobre a
4 nomeação de variáveis. Em Python, o nome de uma variável:
5     - Não pode iniciar com número
6     - Não pode conter espaços e caracteres especiais como @,$,#,!, etc.
7     - Não podem ser palavras reservadas (que já possuem um significado dentro da
8       linguagem) como int, if, in, etc.
9     - Diferencia letras maiúsculas e minúsculas, ou seja, nome_variavel é
10      diferente de nome_Variavel.
11     - Por convenção (não é algo obrigatório mas é adotado como padrão) os nomes
12      seguem um formato chamado snake_case, onde todas as letras são minúsculas
13      e as palavras são separadas por underline/barra baixa (_).
14 '''
15
```



interacao_com_o_usuario.py x

Interação com o usuário

'''

- Agora vamos, finalmente, criar nosso primeiro programa Python!
- Junto disso vamos aprender nossas duas primeiras funções (falaremos mais sobre funções em uma futura aula)

'''

Função print()

'''

- Provavelmente a função mais popular da linguagem e é usada para escrever textos na tela durante a execução do programa
- Para usar, basta escrever `print("string com o texto que deseja exibir")`
- Exemplo prático: "Hello, World!"
 - Crie um novo arquivo Python chamado `hello.py`
 - Use a função `print()` para escrever "Hello, World!" na tela
 - Execute o programa e veja o resultado
 - Desafio extra: Substitua a string dentro da função por uma variável de forma que o resultado seja o mesmo

'''

Função input()

'''

- Funciona de forma similar a função print(), mas também recebe uma entrada do usuário e trava a execução do programa até que ele digite algo.
- Seu uso é idêntico ao print(), basta digitar input("String que será exibida")
- Podemos armazenar o dado que o usuário digitar em uma variável da seguinte forma:
 - entrada_usuario: str = input("Digite algo: ")
- O dado recebido sempre será uma string, portanto, se quisermos usá-lo como um outro tipo devemos converter ele
 - numero_usuario: int = int(input("Digite um número: "))

'''

interacao_com_o_usuario.py x

```
1
2
3
4 '''
5     - Exemplo prático: "Olá, usuário!"
6       - Crie um novo arquivo Python chamado ola.py
7       - Use a função input() para ler o nome do usuário e guarde esse nome em
8         uma variável
9       - Por fim, use a função print() para exibir "Olá, *nome do usuário*"
10      - Dica: Use concatenação de strings para exibir o nome
11 '''
12
13
14
15
```



interacao_com_o_usuario.py x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

Agora vamos para os exercícios!